Machine Learning-Based Classification for Sentiment Analysis of IMDb Reviews

CineSentinel AI — IMDb Sentiment Intelligence Lab · UE24CS352A — Machine Learning

Team: [Team member 1 — SRN] · [Team member 2 — SRN] · [Team member 3 — SRN]

1. Problem statement

Given the text of an IMDb movie review, classify the reviewer's opinion as **positive** or **negative**. This is document-level binary sentiment classification. The work follows the methodology of Wu and Shin [1], who compare six classifier families on this task.

2. Objectives

- Build a reproducible pipeline: data acquisition, text cleaning, feature extraction, training and evaluation.
- Train and compare six classifiers on identical features, using standard metrics on a held-out test set.
- Provide an interactive application for single-review prediction, model comparison and batch scoring of CSV files.

3. Dataset

The Stanford Large Movie Review Dataset [2] contains 50,000 labelled reviews in an official split of 25,000 training and 25,000 test reviews, each exactly half positive and half negative (rating ≥ 7 is positive, ≤ 4 is negative). The median review has 174 words. The official split is used unchanged and the test split is used only for the final evaluation. The public dataset contains 123 test reviews (0.49%) that also occur in the training split; they were left in place, so test accuracy may be very slightly optimistic. The files were downloaded from a HuggingFace mirror and all review–label pairs were verified to be identical to the official Stanford archive.

4. Preprocessing and feature extraction

Each review is stripped of HTML, lowercased, stripped of punctuation and digits, tokenized, filtered for English stop words and lemmatized (NLTK). Reviews are then converted to **TF-IDF** vectors over unigrams and bigrams, limited to the 50,000 most frequent terms (min. document frequency 5, sublinear term frequency). The vectorizer is fitted on the training split only, and the resulting matrix is shared by all six models.

5. Classifiers

Model	Settings
Logistic Regression	L2 penalty; C from {0.01, 0.05, 0.25, 0.5, 1} by 3-fold CV → C = 1
Linear SVM	L2 penalty; same C grid → C = 0.25; Platt-calibrated
Multinomial Naive Bayes	Laplace smoothing = 1
Random Forest	100 trees, Gini criterion, no depth limit
Gradient Boosting	270 trees (chosen on validation data), learning rate 0.1
Neural network (MLPClassifier)	hidden layers (30, 30, 20, 10, 10), logistic activation, Adam, early stopping

Settings follow §3.3 of [1] except the Gradient Boosting tree count (Section 7). All model selection used training data only: cross-validation for C, and a held-out 10% of the training split for the neural network's early stopping.

6. Implementation

Python 3.11 with scikit-learn [3], NLTK, Streamlit and Plotly, running on a laptop CPU with no GPU or paid service. Training and evaluation are command-line modules that write model files and metrics files; the five-page Streamlit dashboard reads those files, so no figure is typed into the application. The project has an automated test suite (pytest).

7. Results

Measured once on the 25,000-review test split. The positive class is "positive"; precision, recall and F1 are for that class, and Macro-F1 averages both classes.

Model	Accuracy	Precision	Recall	F1	Macro-F1	TP / FP / FN / TN	Train (s)
Deep Neural Network (MLP)	0.8948	0.8944	0.8952	0.8948	0.8948	11,190 / 1,321 / 1,310 / 11,179	238.7
Linear SVM	0.8938	0.8988	0.8874	0.8931	0.8938	11,093 / 1,249 / 1,407 / 11,251	3.4
Logistic Regression	0.8922	0.8904	0.8946	0.8925	0.8922	11,182 / 1,376 / 1,318 / 11,124	8.4
Multinomial Naive Bayes	0.8633	0.8787	0.8430	0.8605	0.8633	10,537 / 1,454 / 1,963 / 11,046	0.1
Random Forest	0.8560	0.8684	0.8391	0.8535	0.8560	10,489 / 1,589 / 2,011 / 10,911	72.4
Gradient Boosting	0.8441	0.8265	0.8710	0.8482	0.8440	10,888 / 2,286 / 1,612 / 10,214	476.7

Deep Neural Network (MLP) has the highest measured accuracy (89.48%), but the top three (Deep Neural Network (MLP), Linear SVM and Logistic Regression) are within 0.25 percentage points of one another. Each model was trained once, so this gap is too small to rank them reliably. Gradient Boosting is the weakest model (84.41%).

With the paper's 100 trees, Gradient Boosting scored 81.07% on the test split. This was investigated rather than accepted: a control model trained on shuffled labels scored 51.0% (chance), ruling out leakage or misaligned labels, and even a 300-tree model used only 839 of 50,000 features, so the 100-tree model was simply too small. The tree count was therefore chosen on a validation split of the training data (20,000 fit / 5,000 validation), where accuracy rose from 80.34% at 100 trees to 83.24% at 270. The final 270-tree model, evaluated once, scores 84.41% on the test split.

A side experiment compared binary, word-count and TF-IDF features with 1-, 1–2- and 1–3-grams for the three fast classifiers, on the same validation split. Validation accuracy ranged from 85.2% to 89.4%; the best was Linear SVM with TF-IDF 1-3-grams. TF-IDF gave the highest accuracy for every classifier and n-gram range, which differs from the paper, where binary features were best. Classifier settings were not re-tuned per representation.

8. Conclusion

All six classifiers were implemented, trained and evaluated on the official test split. The two linear models and the neural network reach about 89% accuracy, while Naive Bayes and the tree ensembles are lower; this is the same overall pattern reported in [1]. For this task a linear model offers the best balance: it matches the neural network's accuracy with a small fraction of the training time and remains interpretable.

9. Limitations

- Not an exact reproduction of [1], which uses 3-gram features and does not publish its vocabulary size, preprocessing code or random seeds.
- Each model was trained once; no confidence intervals or repeated seeds are reported.
- Bag-of-words features ignore word order, so sarcasm and long-range negation are handled poorly.
- The feature-representation experiment covers three classifiers on validation data, not all six on the test split.

References

[1] C.-L. Wu and S.-L. Shin, "Machine Learning based classification for Sentimental analysis of IMDb reviews," Stanford University, CS229 project report.

[2] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng and C. Potts, "Learning Word Vectors for Sentiment Analysis," *Proc. 49th Annual Meeting of the ACL: Human Language Technologies*, 2011. Dataset: https://ai.stanford.edu/~amaas/data/sentiment/

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.